

```
In [4]: #Bubble Sort
#Space complexity O(1)
#Time complexity a)Best Case: O(n**2)
#           b)Worst Case:O(n)
class solution:
    def BubbleSort(self,arr):
        n=len(arr)
        IsSwap=False
        for i in range(n):
            for j in range(n-i-1):
                if arr[j+1]<arr[j]:
                    temp=arr[j]
                    arr[j]=arr[j+1]
                    arr[j+1]=temp
                    IsSwap=True
            if not IsSwap:
                break
        return arr
obj=solution()
r=obj.BubbleSort([5,1,4,2,8])
print(f'Sorted Array is:{r}')
```

Sorted Array is:[1, 2, 4, 5, 8]

```
In [ ]: #Insertion Sort
#Space complexity O(1)
#Time complexity a)Best Case: O(n**2)
#           b)Worst Case:O(n)
class solution:
    def InsertionSort(self,arr):
        n=len(arr)
        for i in range(1,n):
            key=arr[i]
            j=i-1
            while j>=0 and arr[j]>key:
                arr[j+1]=arr[j]
                j-=1
            arr[j+1]=key
        return arr
obj=solution()
r=obj.InsertionSort([5,1,4,2,8])
print(f'Sorted Array is:{r}')
```

Sorted Array is:[1, 2, 4, 5, 8]

```
In [ ]: #Selection Sort
#Space complexity:O(1)
#Time Complexity:waste case->O(n**2)
#           #Best Case->O(n**2)
class solution:
    def SelectionSort(self,arr):
        n=len(arr)
        for i in range(n):
            min_index=i
            for j in range(i+1,n):
                if arr[j]<arr[min_index]:
                    min_index=j
            temp=arr[i]
            arr[i]=arr[min_index]
```

```

        arr[min_index]=temp

    return arr
obj=solution()
r=obj.SelectionSort([5,1,4,2,8])
print(f'Sorted Array is:{r}')

```

Sorted Array is:[1, 2, 4, 5, 8]

```

In [ ]: #Merge Sort--> Recursive process
#Space complexity:O(n)
#Time Complexity:O(n log n)
class solution:
    def Merge(self,arr,l,mid,r):
        a=[]
        b=[]

        for i in range(l,mid+1):
            a.append(arr[i])
        for i in range(mid+1,r+1):
            b.append(arr[i])

        i,j,k=0,0,1
        while k<=r:

            if j==len(b):
                arr[k]=a[i]
                i+=1
                k+=1
            elif i==len(a):
                arr[k]=b[j]
                j+=1
                k+=1

            elif a[i]<b[j]:
                arr[k]=a[i]
                i+=1
                k+=1
            else:
                arr[k]=b[j]
                j+=1
                k+=1

    def MergeSort(self,arr,l,r):
        #Base case
        if l>=r:
            return

        #Recurive case
        mid=(l+r)//2
        self.MergeSort(arr,l,mid)
        self.MergeSort(arr,mid+1,r)

        self.Merge(arr,l,mid,r)

    def SortArray(self,arr):
        n=len(arr)
        self.MergeSort(arr,0,n-1)

```

```

        return arr

obj=solution()
r=obj.SortArray([5,1,4,2,8])
print(f'Sorted Array is:{r}')

```

Sorted Array is:[1, 2, 4, 5, 8]

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click [here](\"https://aka.ms/vscodeJupyterKernelCrash\") for more info.

View Jupyter [log](\"command:jupyter.viewOutput\") for further details.

In []: *#Quick sort-->Here also recursive process*

```

#Space complexity->O(1)
#Time Complexity->O(n**2)
        #avg TC->O(n Log n)
class solution:

    def Partition(self,arr,l,r):
        key=arr[r]
        start=l
        n=len(arr)

        for i in range(l,r+1):
            if arr[i]<=key:
                temp=arr[i]
                arr[i]=arr[start]
                arr[start]=temp
                start+=1
        return start-1

    def QuickSort(self,arr,l,r):
        #Base class
        if l>=r:
            return

        #Recursive Case
        p=self.Partition(arr,l,r)

        self.QuickSort(arr,l,p-1)
        self.QuickSort(arr,p+1,r)

    def SortArray(self,arr):
        n=len(arr)
        self.QuickSort(arr,0,n-1)
        return arr

obj=solution()
r=obj.SortArray([5,1,4,2,8])
print(f'Sorted Array is:{r}')

```

Sorted Array is:[1, 2, 4, 5, 8]